V2 User Profile Dataset — Schema Guide

Dataset: `yufan/user_profile_dataset` · **Configs:** the 8 `V2_*` configs · **Teacher:** GPT-5.6

For every config this guide shows what the model reads (**Input**) and what it must return (**Output**).

1. Overview

Config	Task	Train	Test
`V2_User_Profile_L1_gpt56`	Raw activities → interests	267,810	5,378
`V2_User_Profile_L1_Negative_gpt56`	Negative feedback → rejected topics	557,734	11,184
`V2_User_Profile_L2_gpt56`	Existing profile + new interests → merge / add decisions	259,918	5,199
`V2_User_Profile_L3_Commercial_gpt56`	Interests → commercial label, brands, search queries	34,722	1,712
`V2_User_Profile_L3_Persona_gpt56`	Interests → persona + category	19,258	966
`V2_User_Profile_L4_Biography_gpt56`	Interests + facts → life stage, attributes, biography	19,482	973
`V2_User_Profile_L4_CommercialPreference_gpt56`	Commercial interests → shopping preferences	16,311	811
`V2_User_Profile_L4_Hyper_gpt56`	Commercial interests + context → ad missions + new queries	11,584	567

2. Row format (all configs)

```
{
  uid: string,             // 16-char hex user ID (L1 Negative: hash of the row's actions, the data has no user ID)
  messages: [
    { role: "user",      content: PROMPT + "\n\nInput:\n" + INPUT },
    { role: "assistant", content: OUTPUT }          // minimized JSON
  ],
  source: "GPT56",
  domain: string,          // see the table below
  think_type: "none",
  language?: string        // see the table below
}
```

- One fixed prompt per config, no system message, always one user turn and one assistant turn.
- `OUTPUT` always parses with `json.loads`.

How to read a row:

```python
import json

prompt, _, raw_input = row["messages"][0]["content"].rpartition("\nInput:\n")
output = json.loads(row["messages"][1]["content"])
if row["domain"] in ("maiprofile-layer1-combined", "maiprofile-layer1-negative"):  # JSONL: one object per line
    input_data = [json.loads(line) for line in raw_input.split("\n")]
else:
    input_data = json.loads(raw_input)
```

Config	`domain`	`language` column	Input format
L1	`maiprofile-layer1-combined`	content language (= output `predicted_content_locale`)	JSONL
L1 Negative	`maiprofile-layer1-negative`	market: `en` or `glb`	JSONL
L2	`maiprofile-layer2-combined`	*(no column)*	JSON
L3 Commercial	`maiprofile-layer3-commercial`	input `query_language`	JSON
L3 Persona	`maiprofile-layer3-persona`	always `en`	JSON
L4 Biography	`maiprofile-layer4-biography`	always `en`	JSON
L4 Commercial Preference	`maiprofile-layer4-commercial-preference`	always `en`	JSON
L4 Hyper	`maiprofile-layer4-hyper`	input `query_language`	JSON

3. How the layers connect

```
L1  raw activities -> interests
 `-- L2  merge new interests into the user's profile
     |-- L3 Persona       persona + category for each interest
     `-- L3 Commercial    commercial label, brands, queries for each interest
          `-- L4 Biography                 life stage, user attributes, biography
               `-- L4 Commercial Preference  shopping preferences
                    `-- L4 Hyper              ad missions + new queries

L1 Negative  negative feedback -> rejected topics   (standalone: no user ID to join with the other layers)
```

4. Shared types and notation

```typescript
// Source: activity source. The last 7 values are rare (27 occurrences, only in L3 Commercial / L4 Biography inputs)
type Source = "MSN" | "Edge" | "Uet" | "Bing" | "Ads" | "ChromeImports" | "Shopping" | "Linkedin"
            | "YouTube" | "Amazon" | "Xbox" | "Facebook" | "Google Play" | "Walmart" | "Yahoo";
type Lang   = "en" | "de" | "fr" | "es" | "ja" | "pt" | "nl" | "it" | "sv" | "pl" | "zh-Hans" | "zh-Hant";
type Level  = "high" | "medium" | "low";
type Scale  = "Low" | "Medium-Low" | "Medium" | "Medium-High" | "High";
type Signal = "strong" | "moderate" | "weak";
```

- `field?` = optional field; `~N%` = how often it is present.
- All counts (`1–39, median 10`) and percentages are measured on every row of the config (train + test).

L1 — Interest Extraction

`V2_User_Profile_L1_gpt56` · Find the user's interests in one window of activities.

Input — JSONL, one activity per line (1–200 lines, median 37)

```
{ idx: int, date: "YYYY-MM-DD", source: Source, action: string }   // action: page title, query, or article title
```

Output

```
{
  predicted_content_locale: Lang | "mix",   // main language of the activities
  interests: [{                             // 0-39, median 10 (0.3% of rows have none)
    interest_name: string,
    topics: [{                              // 1-53, median 2
      topic: string,
      source: Source[],                     // 1-6
      evidence: int[]                       // input idx
    }],
    actual_activity: string,                // one factual sentence
    inferred_intent: string                 // the intent one step above the activity
  }]
}
```

L1 Negative — Rejected Topics

`V2_User_Profile_L1_Negative_gpt56` · Find the topics the user rejected in one batch of negative feedback.

Input — JSONL, one signal per line (1–148 lines, median 1; 74% of rows have one signal; no dates)

```
{
  idx: int,                       // 0..n-1
  source: "MSN" | "Ads",
  detailed_source: "ContentUserFeedback" | "AdUserFeedback",
  action: string                  // MSN: "Downvote | <content title>"
                                  // Ads: "NotInterested | <advertiser> | <ad title>"
}
```

Output

```
{
  negative_interests: [{          // 1-103, median 1
    keywords: string,             // the rejected topic: 1-8 lowercase words (median 4), any language
    evidence: int[],              // input idx it is based on (95% cite one signal)
    source: ("MSN" | "Ads")[],    // sources of the cited signals
    detailed_source: string,      // detailed source of the cited signals
    signal_type: string[],        // lowercase action prefixes: "downvote" | "notinterested"
    reliability_score: float      // 0-1 (observed 0.72-0.99, median 0.96)
  }]
}
```

Notes

- Every input signal is cited by at least one interest.
- `MSN` always pairs with `ContentUserFeedback` and `downvote`; `Ads` with `AdUserFeedback` and `notinterested` (one interest has the combined `signal_type` `"repeatedad~inappropriatead~notinterested"`).
- `keywords` names a topic, never a reaction such as "offensive"; 94% are plain ASCII (mostly English).

L2 — Profile Merge

`V2_User_Profile_L2_gpt56` · Merge each new interest into the user's existing profile, or add it as new.

Input

```
{
  // existing profile, 0–275 interests (median 39)
  snapshot: [{ interest_name: string, actual_activity: string, topics: string[] }],
  // new interests, 1–95 (median 10)
  delta:    [{ interest_name: string, actual_activity: string, topics: string[] }]
}
```

Output — one decision per `delta` interest, in the same order (66 rows, 0.02%, list them in another order)

```
{
  decisions: [
      { action: "merge",                    // ~60%
        delta_interest_name: string, snapshot_interest_name: string,
        merged_interest_name: string, merged_actual_activity: string, merged_inferred_intent: string }
    | { action: "add",                      // ~40%
        delta_interest_name: string, actual_activity: string, inferred_intent: string }
  ]
}
```

L3 Commercial — Commercial Classification

`V2_User_Profile_L3_Commercial_gpt56` · Decide whether advertisers could target each interest, and list its entities and search queries.

Input

```
{
  interests: [{                             // 1–30, median 24
    interest_name: string,
    actual_activity: string,
    sources: Source[],
    topics: [{ topic: string, actions: string[] }]   // actions: raw activity texts (0–3)
  }],
  query_language: Lang
}
```

Output — one entry per input interest

```
{
  interest_commercial: [{
    interest_name: string,
    commercial: boolean,                                        // true ~75%
    commercial_score: "high" | "medium" | "low" | null,
    intent_funnel_stage: "discovery" | "research" | "consideration" | "purchase" | "post-purchase" | null,
    brands: string[],
    retailers: string[],
    products: string[],
    predicted_queries: string[]                                 // ≤ 3, in query_language
  }]
}
```

Notes

- When `commercial` is `false`: `commercial_score` and `intent_funnel_stage` are `null`, and all lists are `[]`.

L3 Persona — Persona & Category

`V2_User_Profile_L3_Persona_gpt56` · Write a persona and a category path for each interest.

Input

```
{
  interests: [{                // 1–187, median 43
    interest_name: string,
    actual_activity: string,
    inferred_intent: string,
    topics: string[]
  }]
}
```

Output — one entry per input interest

```
{
  interest_personas: [{
    interest_name: string,
    category: string,          // "/" path, broad → specific,
                               //   e.g. "/Technology/Artificial Intelligence/AI Assistants"
    persona: string
  }]
}
```

L4 Biography — Life Stage, Attributes, Biography

`V2_User_Profile_L4_Biography_gpt56` · Infer the user's life stage and attributes, and write a short biography.

Input

```
{
  declared_attributes: {                    // {} in 65% of rows
    age_group?: "18 - 24" | "25 - 34" | "35 - 49" | "50 - 65" | "65+",   // ~32%
    gender?: "Male" | "Female"                                           // ~26%
  },
  facts: {                                  // {} in 55% of rows
    market?: string,                        // ~28%, e.g. "en-US"
    location_data?: {                       // ~30%
      current_location: string,
      place_frequency:  [{ place: string,  count: int }],
      region_frequency?: [{ region: string, count: int }],   // ~97% of location_data
      recent_visits?: string[]                               // ~53% of location_data
    },
    linkedin_profile?: {                    // ~16%; sparse fields, each a string or string[]
      profile_geo?: string,                 // 99.9% of linkedin_profile
      industries?, degree?, degrees?, fieldOfStudy?, areas?, titles?, current_roles?,
      yearsOfExperience?, experience_band?, seniorities?, functions?, companies?, current_company?
    }
  },
  interests: [{                             // 1–190, median 44
    interest_name: string,
    actual_activity: string,
    inferred_intent: string,
    persona: string,                        // from L3 Persona
    confidence_score: float,
    count: int,
    first_detect_date: "YYYY-MM-DD",
    last_detect_date: "YYYY-MM-DD",
    source: Source[],
    topics: string[],
    predicted_queries?: string[],           // ~66%, from L3 Commercial
    products?: string[],                    // ~51%
    brands?: string[]                       // ~44%
  }]
}
```

Output

```
type AttrSource = "declared" | "behavioral_inference" | "location_data" | "linkedin" | "unknown";
type Attr = { evidence: string[], value: string, confidence: Level, source: AttrSource };

{
  life_stage: {
    value: "single" | "married" | "parenting" | "caregiving" | "job_seeking" | "new_grad" | "retirement"
         | "unknown",
    confidence: Level,
    evidence: string[]                      // 1-3
  },
  user_attributes: {
    age_group: Attr & { signals?: AgeSignals },
                                            // "13 - 17" | "18 - 24" | "25 - 34" | "35 - 49" | "50 - 65" | "65+"
                                            //   | "Unknown"
    gender: Attr & { lean: { male: string[], female: string[] } },   // "Male" | "Female" | "Unknown"
    estimated_income_level: Attr,           // "High" | "Medium" | "Low" | "Unknown"
    home_location: Attr,                    // place name | "Unknown"
    occupation: Attr,                       // occupation category | "Unknown"
    company: Attr,                          // company name | "Self-Employed" | "Unknown"
    career_stage: Attr,                     // "Student" | "Entry Level" | "Mid-Level Professional"
                                            //   | "Senior Professional" | "Manager" | "Director/Executive"
                                            //   | "Business Owner/Self-Employed" | "Retired"
                                            //   | "Not in Workforce" | "Unknown"
    education_area: Attr                    // field of study | "Unknown"
  },
  biography: string                         // 2-3 sentences
}

type AgeSignals = {                         // present ~68%
  later_life_status: boolean,
  later_life_clusters: ("home_garden" | "leisure_travel_clubs" | "savings_income" | "heritage_hobbies"
                      | "classic_media" | "health_aging" | "grandchildren")[],
  children_content: boolean,
  own_current_study: boolean,
  own_high_school: boolean,
  household_signals: boolean,
  news_heavy: boolean,
  youth_culture_top5: boolean,
  early_adult_events: string[],             // e.g. "first-home buying"
  linkedin_min_age?: int | null
};
```

Notes

- `gender.lean` entries look like `"Football (moderate)"`.

L4 Commercial Preference — Shopping Preferences

`V2_User_Profile_L4_CommercialPreference_gpt56` · Summarize how the user shops.

Input

```
{
  life_stage: { value: string, confidence: Level, evidence: string[] },   // L4 Biography output
  commercial_interests: [{                  // 1-114, median 27
    interest_name: string,
    persona: string,
    brands: string[],
    retailers: string[],
    products: string[],
    topics: [{ topic: string, evidence: string[] }]   // evidence: raw activity texts
  }],
  non_commercial_interest_names: string[]
}
```

Output

```
type Detail<K extends string, V> = { area: string, evidence: string, signal_strength: Signal } & { [key in K]: V };

{
  discount_sensitivity:  { value: Scale | "Unknown", details: Detail<"sensitivity", Scale>[] },
  price_sensitivity:     { value: Scale | "Unknown", details: Detail<"sensitivity", Scale>[] },
  subscription_affinity: { value: Scale | "Unknown", details: Detail<"affinity", Scale>[] },
  affinity: {
    shopping: {
      product_categories: [{ category: AdsVertical, description: string }],
      shopper_type: { value: ShopperType[], details: Detail<"type", ShopperType>[] }
    },
    dining: {
      restrictions: { value: Diet[], details: Detail<"restriction", Diet>[] }   // value ["Unknown"] when no signal
    }
  }
}

type AdsVertical = "HomeGarden" | "ComputersConsumerElectronics" | "RetailersGeneralMerchandise" | "TravelTourism"
                 | "Finance" | "Apparel" | "Vehicles" | "InternetTelecom" | "Health" | "BeautyPersonalCare"
                 | "SportsFitness";          // Ads Vertical L1
type ShopperType = "ResearchDriven" | "DealDriven" | "BrandRetailerLoyal" | "ValueDriven"
                 | "NoveltySeeker" | "ConvenienceDriven" | "PremiumDriven";
type Diet        = "Unknown" | "Vegan" | "Vegetarian" | "No Gluten" | "Low-Carb" | "Keto" | "Halal" | "Dairy-Free"
                 | "Kosher" | "Pescatarian" | "Paleo" | "No Nuts";    // the last 4 are rare (25 occurrences)
```

Notes

- When a `value` is `"Unknown"`, its `details` is `[]`.

L4 Hyper — Ad Missions & Queries

`V2_User_Profile_L4_Hyper_gpt56` · Turn the user's commercial interests into missions for ads retrieval, each with new search queries.

Input — a JSON object whose first four fields are **strings**

```
{
  personal_context: string,        // Markdown list or "": Market, Current location, Age group, Gender,
                                   //   Recent visits
  professional_context: string,    // Markdown list, "" in ~88%: Current company, Current roles, Functions,
                                   //   Industries, Seniorities, Years of experience, Degrees, Education areas
  commercial_preferences: string,  // a JSON string: parse it again (format below)
  commercial_interests: string,    // Markdown, one block per interest (format below)
  query_language: Lang
}
```

`commercial_preferences`, after parsing — a short form of the L4 Commercial Preference output (`details` replaced by `confidence_score`):

```
{
  discount_sensitivity?:  { value: Scale, confidence_score: float },
  price_sensitivity?:     { value: Scale, confidence_score: float },
  subscription_affinity?: { value: Scale, confidence_score: float },
  affinity: {
    shopping: {
      product_categories?: [{ category: string, description: string }],   // missing in 1 row
      shopper_type?: { value: ShopperType[], confidence_score: float }
    },
    dining?: { restrictions: { value: Diet[], confidence_score: float } }
  }
}
```

`commercial_interests` block (in < 0.1% of rows a block misses its Category, Persona, Activity, or Intent line):

```
### Interest 1: <interest_name>
- Category: /A/B/C
- Persona: ...
- Activity: ...
- Intent: ...
- Existing queries:
  - <query>
- Known entities:          (optional)
  - Brands: a, b
  - Retailers: a, b
  - Products: a, b
```

Output

```
{
  missions: [{                                  // 0–16, median 12 (13 rows have none)
    mission_name: string,                       // in query_language
    source_interests: string[],                 // exact input interest names
    scenarios: ("shopping" | "travel" | "technology" | "hobbies" | "learning" | "fitness" | "dining")[],
    predicted_brands: string[],                 // ≤ 3 new brands
    predicted_queries: [{                       // 1–4
      query: string,                            // 1–7 words (median 4), in query_language
      value_type: "explore" | "refine" | "advance",
      source_query_refs: string[],              // Existing queries it improves on
      delta_source: "commercial_preference" | "source_interest" | "personal_context"
                  | "cross_interest" | "professional_context",
      delta_evidence: string,
      decision_change: string
    }],
    enrichment_sources: ("commercial_preference" | "personal_context" | "cross_interest" | "professional_context")[]
  }]
}
```